1.

a) Define Generics in java. How do they help in writing type-safe code? What is Type Inferences in generics? List and explain the restrictions on generics.

Generics :-

→ Generics are set of features that allows to write code independent of data type.

→ Generics allows us to create a single class, interface & method can be used with different types of data (objects).

→ Generic ensure type independence i.e we can provide a type parameter while defining a class / interface / method etc.

→ so that during the actual instantiation. we can specify the actual type. This way also provide code reusability.

→ Java Generics are mostly used with the collections framework of java. The different collections like Linked List, List, Map, HashMap etc.

→ The generics in java is implemented by ~~ang~~ angular bracket '< >' symbol and the type parameter is defined in the ~~p~~ bracket.

→ The type parameter such as 'T' for type, 'E' for element, 'N' for number, 'K' for key, 'V' for value.

→ Generics provide compile-time type checking, which means errors are detected during compilation instead of runtime.

Type Inference means the Java compiler automatically determines the type parameter based on context.

Ex:

```
Box <Integer> b=new Box<>();
```

Restrictions of Generics in Java.

1. No primitive types as type parameters:

cannot directly use primitive types (like int, float, char) as type parameters for generic classes or interfaces.

```
List<int> //not allowed
```

2. No Instruction of type parameters:

cannot create instance of the type parameter directly within the generic class or method.

Ex: `T obj = new T();` // is not allowed.

3. Type erasure & Runtime type information:

Type information is erased at runtime. This means you cannot perform instanceof checks or use reflection to determine the actual type argument at runtime.

Ex: `if (list instanceof List<String>) {...}` is not allowed.

4. Array creation with parameterized types:

cannot create arrays of parameterized types directly.

Ex: `List<String>[] sa=new List<String>[10];` is not allowed.

5. overloading methods with erased types.

b) Demonstrate Generic class (Bounded & Unbounded), Generic Interface a with an example programs. Explain with an example how wildcards can be used in Generics. What is the difference between upper and lower bounded wild cards?

A Generic class is the same as a normal class except thats the class name is followed by a type in angular brackets.

Ex:- class class_Name<T>.
{
}

```
import java.util.*;
public class WithGenerics <T>.
{
    T fn, sn;
    public withGenerics (T a, T b).
    {
        fn = a;
        sn = b;
    }
  public T first( ).
  {
      return fn;
  }
  public T sec( )
  {
      return sn;
} }
public class GenericsDemo.
{
  public static void main (String [] args)
  {
    WithGenerics <String> n= new WithGenerics <String>
      String s=n.first( "Sachin", "Tendulkar");
    System.out.println (s.length());
} }
```

Bounded Types

→ The concept of limiting type parameter upto certain type is known as Bounded Types can bound the Type parameter for a particular range by "extends" keywords.

→ The type parameter can accept any data types (except primitive types).

→ To use Generics for some specific layers / types (such as accept data of Number types) only, then one can use bounded types.

→ In the case of bound types we use the extends keyword.

- Ex:- <T extends A>.

  This means T can only accept data that are subtypes of A.

→ Type parameters can be bounded. Bounded means "restricted", we can restrict the types that a method accepts.

UnBounded Types

As a type parameter can pass any Type and there are no restrictions & hence it is unbounded Type.

Ex:-
```
class Demo<T>
{
    public void m1().
    {
        T x,y;
        System.out.println (x+y);
        System.out.println (x*y);
    }
}
```

on x + y, can perform any ~~arithmetic~~ arithmetic operations. Applying on string + student classes is meaningless.

Example using wildcards:-

```
import java.util.ArrayList;
public class WildCardIssue
{
    public static void main(String[] args)
    {
        ArrayList<String> al=new ArrayList<String>();
        WildCardIssue w=new WildCardIssue();
        w.display(al);
    }
    <T> void display(ArrayList<String> al).
    {
        al.add("Im");
        al.add("from");
        System.out.println(al);
    }
}
```

Ex:- Upper bounded wildcard.

```
import java.util.*;
class Animal {}
class Dog extends Animal {}.
class Cat extends Animal {}.
public class UpWl
{
    public static void printList(ArrayList<? extends Animal> list).
    {
        for (Animal a: list).
        {
            System.out.println(a.getClass().getName());
        }
    }
```

```
public static void main (String[] args).
{
    Arraylist<Dog> d=new Arraylist<>();
     d. add (new Dog[]);
     d. add (new Dog());
  Arraylist<Cat> c=new Arraylist<>();
        c.add (new Cat());
        c.add (new Cat());
  Arraylist <Animal> a1 = new Arraylist<>();
            a1. add (new Dog());
            a1. add (new Cat());
    System. out. println ("printing Dog list:");
          printlist (d);
    System. out. println ("printing cat List:");
          printlist (c);
    System. out. println ("printing Animal");
          printlist (a1);
  }
}.
```